

Pulsaride Medical Dispatch Engine

V1 Spring Boot API, evaluation, and live dashboard

Salmane Sossey Ihssan Ben Labsir

Internship project supervised by M. Lyazid Salihi
Pulsaride Solutions

Oral defense

Executive Summary

Problem

Medical requests must be assigned quickly to available professionals, while keeping the process auditable and measurable.

Delivered V1

A runnable dispatch engine with Spring Boot, PostgreSQL, Redis, Python evaluation scripts, Docker Compose, and a live dashboard.

Key V1 outcome

- Full request lifecycle implemented
- Four dispatch strategies compared
- Robustness and load tested
- Honest limits documented

Project Scope

Functional scope

- Create and prioritize patient requests
- Search available professional slots
- Reserve, propose, accept, refuse, timeout, close
- Keep assignment history and state transitions
- Expose operational metrics

Engineering scope

- Spring Boot API
- PostgreSQL persistence and audit
- Redis queue and locking
- Docker Compose execution
- Python simulation and evaluation

Architecture V1

Runtime view

API	Spring Boot exposes requests, dispatch, availability, metrics, AI mock triage, and dashboard endpoints.
PostgreSQL	Stores professionals, requests, slots, assignments, and state transitions for audit and evaluation.
Redis	Coordinates real-time queueing and slot/request locks to avoid duplicate assignment.
Python evaluator	Loads scenarios, drives the live API, computes metrics, and generates charts.
Docker Compose	Starts PostgreSQL, Redis, and the API in one reproducible environment.

Patient request → Redis priority queue → Dispatch strategy → Slot reservation → Professional response → PostgreSQL audit

Dispatch Lifecycle

Finite-state workflow

PENDING → RESERVED → PROPOSED → ACCEPTED → CLOSED

Refusal or timeout

PROPOSED → REFUSED/FAILED →
PENDING

The request returns to the queue and can be reassigned.

Auditability

Each assignment attempt and each state transition is persisted, so the demo can explain what happened and why.

Why availability is separated from profile data

Professional profile data is stable, while availability is operational and changes during dispatch. V1 stores slots separately so matching can search active availability efficiently.

- AVAILABLE: can receive a proposal
- RESERVED: selected and locked for one request
- BUSY: accepted consultation in progress
- BREAK: unavailable after refusal or timeout
- OFFLINE: manually unavailable
- Redis mirrors active slots and locks selected slots

Dispatch Strategies

ID	Name	Decision rule
S1	Round Robin	Rotates between currently available slots using the last S1 assignment history.
S2	Tag Exact	Requires exact specialty match; useful to expose no-availability cases.
S3	Score Composite	Combines specialty affinity, availability, and professional load.
S4	Lexical IA	Adds deterministic lexical similarity between patient text and professional profile.

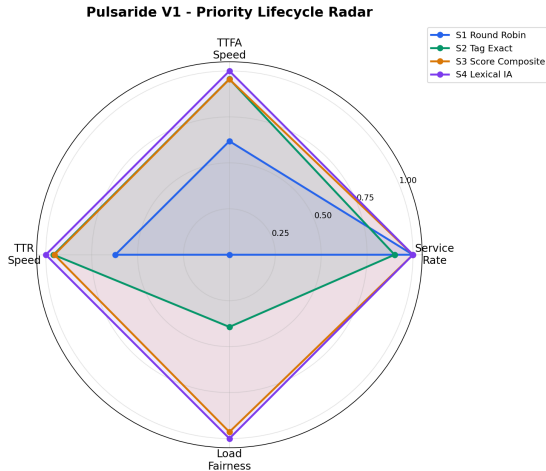
Decision for V1

S3/S4 are the recommended operational strategies; S1 is useful as a fairness baseline and S2 as a strict matching baseline.

Strategy Comparison Results

Strategy	Service	TTFA	TTR	Gini
S1	100%	4 924	5 037	0.54
S2	90%	3 184	3 265	0.43
S3	100%	3 188	3 282	0.27
S4	100%	3 044	3 129	0.26

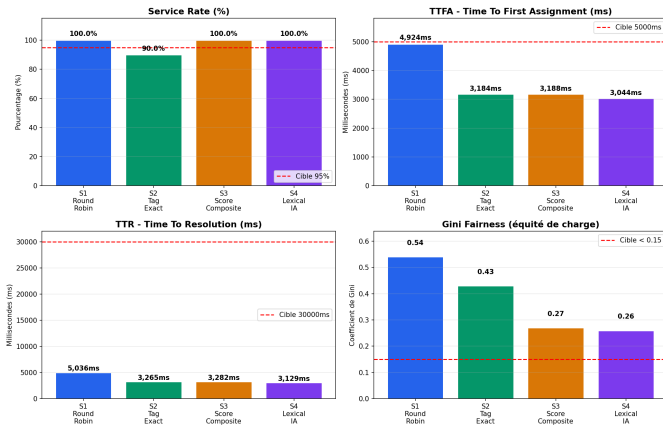
Metrics are produced from the live Spring Boot API with 20 requests, 20 professionals, and fixed seed 42.



Live API run: 20 requests, refusal retry enabled, seed 42. Higher is better.

Comparison Charts

Pulsaride V1 - Comparaison live des stratégies de dispatch
Priority lifecycle: refusals return to queue



Live API run: 20 requests, refusal retry enabled, accept rate 0.85, seed 42.

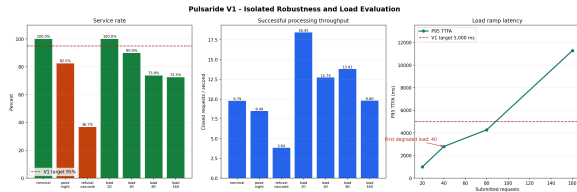
Robustness and Load Evaluation

Latest P4 rerun

- Strategy tested: S3
- Max sustainable load: 20 requests
- Max sustainable throughput: 18.45 closed/s
- First degraded load: 40 requests

Reason for degradation

At 40 requests, service rate drops to 90% because some requests remain PENDING inside the test timeout window.



Delivered dashboard

The dashboard is served directly by Spring Boot at `/dashboard.html`, so no additional frontend service is needed for the V1 demo.

- Live KPIs: service rate, TTFA/TTR
- P95, Gini
- Request flow by status
- Availability by specialty
- Professional status and load
- Auto-refresh every 5 seconds
- Uses existing API endpoints

`http://localhost:8080/dashboard.html`

Automated checks

- Spring Boot test suite passed
- 14 tests, 0 failures
- Round-robin regression test added
- API integration tests cover lifecycle and metrics

Runtime checks

- Docker containers healthy
- Dashboard returns HTTP 200
- Evaluation scripts run against real API
- Charts and JSON reports are versioned

Limits and Honest Interpretation

Known V1 limits

- Gini fairness remains above the ideal target of 0.15.
- Concurrency increases Redis lock contention during high load.
- BREAK slots do not yet return automatically to AVAILABLE.
- S4 is deterministic lexical matching, not a real embedding model yet.

Why this is acceptable for V1

The goal was to produce a reliable, explainable, measurable dispatch engine. The limits are visible in the metrics and can guide V2 priorities.

Demo Plan for the Supervisor

- ➊ Start Docker Compose and show API health.
- ➋ Create one patient request with `curl` or Postman.
- ➌ Dispatch it with S4, then accept and close it.
- ➍ Inspect assignments and state transitions.
- ➎ Open the live dashboard.
- ➏ Show strategy comparison and robustness charts.

Conclusion

What is complete

Implementation, execution environment, evaluation artifacts, dashboard, and technical documentation are complete and pushed to main.

Best next step

Use this V1 as the internship technical deliverable, then package the final written report and defense narrative around measured results and honest limitations.

Thank you